DRAFT - Product Classification

Created By	Lamaan Ball	
Reviewed By		
Published By		

Contents

- Overview
- High Level Design
  - Sequence key
- Low Level Design
  - Projects
  - Ingestion
  - Adobe client
  - Synchronization Layer
  - How to Test
  - Risks
  - Appendix: Included MuleSoft Replacements
  - Appendix: Glossary
  - Appendix: Original diagrams under source control
  - ERP / Backoffice Systems
  - Links and Resources

Overview

This document is the design for **replacing existing MuleSoft** `MUL-PRODUCT-CLASSIFICATION` **behaviour** with a timer-triggered **Classification Function** in the Product Service repo. The business flow is unchanged: SAP Commerce Cloud writes a full **classification feed** to Azure Blob once a day, and that file is imported into **Adobe Classifications**.

Terms in **bold** that link into the glossary are defined there.

There is no HTTP API on this path. Callers do not post to Product Service. The existing Product Service Web API (prices, stock, catalogue reads) stays as it is.

Today the live path is three CloudHub applications:

- `product-feed-papi-adobeFlow` on `sgbd-live-product-feed-papi-v1`, Quartz `0 0 6 * * ? *` (06:00). The analysis sheet names the Runtime Manager schedule `product-feed-papi-adobeFlow_Scheduler`. This timer is recorded as enabled.

- Blob download for this feed is recorded as `sgbd-test-azure-blob-storage-sapi-v1` from path `hybris/export` (test and live). The dependency matrix also lists `stark-live-stark-hr-azure-storage-sapi-v1` on the same integration id.
- `sgbd-live-adobe-cloud-sapi-v1` calls the Adobe Analytics 1.4 classifications API: create import, populate in pages, commit import. Before each Adobe call it requests an IMS access token.

The replacement collapses that chain into one Azure Function. It reads the blob with the Azure SDK and calls the Adobe Analytics **2.0** Classifications API (create job, upload the tab file, commit). It does not call the 1.4 API and it does not re-host the SAPI HTTP routes. Adobe retired the 1.4 APIs on 31 August 2026.

Approved behaviour used for this draft:

- product-feed-papi (adobe classification integration) (version 8)
- adobe-cloud-sapi (version 18)
- stark-hr-azure-storage-sapi (version 27)
- Analysis sheet in this repo: `docs/product-classification/Phase2_PRODUCT-CLASSIFICATION-Analysis_Sheet_v1.0.xlsx`, sheet `PRODUCT-CLASSIFICATION-APIs`. That workbook holds example requests for the live Mule calls. It also contains a client secret. This design cites the path and the non-secret facts only. Do not copy secret cells onto this page.

`product-papi-v1` is listed on the dependency matrix and is **undeployed**. Live pickup is `product-feed-papi-adobeFlow`.

A WSR note (8 October 2025) records this timer as descoped from Phase 1 release 3. That descope means the replacement was left for a later phase. This Phase 1.5 design is that later phase. The CloudHub schedule is still enabled, and the feed stays in scope for the Function projects. STUK-11303 asked for a confirm-or-obsolete check before build; this paragraph is that confirmation.

Jordan Cowley has confirmed that this analytics data is still needed. He has also confirmed that the production import has been broken since August 2026. That lines up with Adobe retiring the 1.4 classifications API on 31 August 2026. The live Mule path still calls those retired methods.

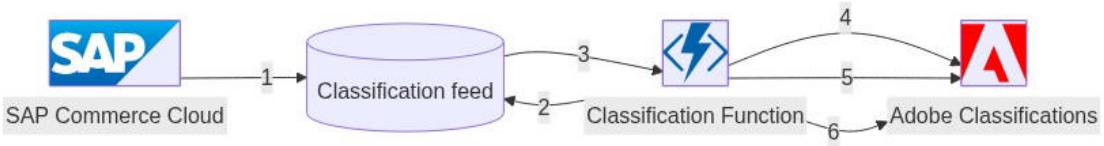
The analytics specification for this file is Adobe Analytics Product Classification File (AEC, version 9). In that space it sits under Solution Design, beside Analytics Solution Design (version 4). Analytics Solution Design is a page tree of the reports: eVars, product list, product details, basket, trolley, checkout, and confirmation. The classification page is the contract for the file that names the product key those reports already collect. On Product Details Pages, the product id comes from the hit

(`digitalData.product[0].productID`). **Product Title** and **Category** are sourced as **Classification Upload**.

That page also describes an older Station10 transport: a UTF-8 tab file uploaded to Adobe FTP, then an empty `.fin` file so Adobe will import it. This Function does not use FTP. It keeps the API import. The part that still applies is the column contract, and the check that those columns show up on the product reports.

High Level Design

This picture is the whole replacement. SAP Commerce Cloud writes the file. Once a day the Classification Function reads it and sends it to Adobe Classifications. The Product Service Web API is not on this path.



Mermaid source (paste into mermaid.live)

```
%% confluence-attachment: hld-product-classification.png
%%{init: {"theme": "default", "flowchart": {"htmlLabels": true, "curve": "basis"}}}%%
flowchart LR
  sapcc@{ img: "https://api.iconify.design/logos/sap.svg", label: "SAP Commerce Cloud", pos: "b", h: 48, constraint: "on" }
  blob[("Classification feed")]
  fn@{ img: "https://api.iconify.design/simple-icons/azurefunctions.svg?color=%230062AD", label: "Classification Function", pos: "b", h: 48, constraint: "on" }
  adobe@{ img: "https://api.iconify.design/simple-icons/adobe.svg?color=%23FF0000", label: "Adobe Classifications", pos: "b", h: 48, constraint: "on" }

  sapcc -->|"1"| blob
  fn -->|"2"| blob
  blob -->|"3"| fn
  fn -->|"4"| adobe
  fn -->|"5"| adobe
  fn -->|"6"| adobe
```

Sequence key

Step	What happens
1	SAP Commerce Cloud writes the full classification feed to Azure Blob. The Function does not delete that blob. SAPCC overwrites it the next day.
2	The Classification Function timer starts (06:00 UTC).

3	The Function reads the blob. A missing or empty file stops the run. Adobe is not called.
4	The Function creates a 2.0 import job and keeps the **job id**.
5	The Function uploads the tab file to that job.
6	When the upload succeeded, the Function commits the import. A failed upload is not committed.

Low Level Design

This section is how we implement the replacement. The work lives in three new projects, described below. They are not in the solution yet. The **Classification Function** is a thin timer. Blob access, the tab file, and the Adobe 2.0 calls live in the processor project.

There is no database in this slice. Mule does not persist the feed, and the approved LLD says a full daily feed has no replay. Application Insights is the run record: blob name, file size, job id, and outcome. The feed payload and the Adobe token are not logged at Information.

Projects

Three new projects, all on .NET 10: a class library for the work, a Functions host for the timer, and integration tests against the library.

Project	Role
`STUK.ProductService.Processor`	.NET 10 class library. Read the blob, unzip when needed, obtain a token, create / upload / commit / status against Adobe Classifications 2.0.
`STUK.ProductService.Processor.Functions`	.NET 10, Azure Functions v4 isolated worker. One timer function calls the processor. Cron and the disable flag come from configuration.
`STUK.ProductService.IntegrationTests`	.NET 10 xUnit project against the processor. A stub Adobe API covers create, upload, commit, and status. A fixture blob stands in for SAPCC.

`STUK.ProductService.Processor.Functions` references `STUK.ProductService.Processor`. Integration tests reference the processor, not the Functions host, so they do not need the timer to fire.

The timer logs the run, then awaits one service method. Suggested names:

Setting or type	Value
Function name	`AdobeClassificationProcessor`
Cron app setting	`AdobeClassificationProcessorCronExpression`
Default NCRONTAB	`0 0 6 * * *`
Disable flag	`DisableAdobeClassificationProcessor`
Global company id	`AdobeClassificationGlobalCompanyId`
Dataset id	`AdobeClassificationDatasetId`
Report suite, test	`saintgobaindistributiondev` (selects the test dataset; not sent on `createApiJob`)
Report suite, live	`saintgobaindistributionprod` (selects the live dataset; not sent on `createApiJob`)

Azure Functions cron is six-field NCRONTAB in UTC. `0 0 6 * * *` is 06:00 UTC, which is the 06:00 GMT schedule on the Mule timer (`0 0 6 * * ? *`). Set the Function app timezone only if a later decision wants local time instead of GMT.

Ingestion

The Function reads Azure Blob directly. It does not call a storage SAPI. `stark-hr-azure-storage-sapi` stays in place because `MUL-CONCUR` still uses it. The analysis sheet records this feed's download as `sgbd-test-azure-blob-storage-sapi-v1` with path `hybris/export` for test and live.

The file is a single full feed, `adobe-products-feed.tab`, every day at 06:00 GMT. The sheet says the file is not deleted and the filename is not changed. The approved process LLD

also says the storage call can return a zip. Container name is still environment-specific (SAP cloud storage S2 test, P2 prod).

Setting	Default in this draft
Container	Environment-specific. Confirm against SAP cloud storage S2 (test) and P2 (prod).
Subfolder	`hybris/export`
Blob name	`adobe-products-feed.tab`

When the blob name ends in `.zip`, or the bytes are a zip, unzip and take the `.tab` member (default member name `adobe-products-feed.tab`). The Function does not split the file into 1.4 populate pages.

The feed is a SiteCatalyst SAINT file. Evidence is `docs/product-classification/sample/adobe-products-feed.tab` (about 11 MB, about 51,000 data rows, dated 5 October 2026 in the file header). The first lines are `## SC` pre-process headers. Mule does not send those lines to Adobe: it separates the column header (`Key` and the classification names) from the data rows. The Function drops every line that starts with `## SC` and uploads the remainder (the `Key` row plus data rows). A 2.0 upload of the raw blob would send the SAINT preamble as well.

A missing blob, a zip with no tab member, a header-only file, or zero data rows after the `## SC` lines are dropped is an empty feed. The run stops. Create, upload, and commit are not called. The blob is left for SAPCC to overwrite.

The Function does not delete or rename the blob. The analysis sheet and the approved LLD both say the source overwrites the file daily.

Adobe client

The processor calls the Adobe Analytics **2.0** Classifications API on `https://analytics.adobe.io/api`. The C# client can be generated with NSwag from the OpenAPI 3.0 file classification.json (API reference). `uploadImportFile` has no request body in that file, so the upload call is written by hand around the generated client.

The Mule SAPI calls the 1.4 methods on `api.omniture.com`. Those endpoints, and WSSE authentication, were retired on 31 August 2026 and Adobe states they are no longer accessible (end-of-life). This design does not call them. The 2.0 file-import path is the supported replacement for a `.tab` feed: create a job, upload the file, commit the job (migration guide).

Call	Adobe path	1.4 behaviour it replaces
Create	`POST /{globalCompanyId}/classifications/job/import/createApiJob/{datasetId}`	`Classifications.CreateImport`
Upload	`POST /{globalCompanyId}/classifications/job/import/uploadFile/{jobId}`	`Classifications.PopulateImport` (the whole file, not 20,000-row pages)
Commit	`POST /{globalCompanyId}/classifications/job/import/commitApiJob/{jobId}`	`Classifications.CommitImport`
Status	`GET /{globalCompanyId}/classifications/job/{jobId}`	`POST /analytics/import/status`, which calls `Classifications.GetStatus`

`globalCompanyId` and `datasetId` come from configuration, one pair per environment. The 2.0 API does not create classification datasets. The dataset already exists in the Adobe Analytics UI. The analysis sheet shows what that dataset corresponds to on 1.4, and these values are the same for the shape of the feed even though they are not fields on `createApiJob`:

1.4 create field	Value in the analysis sheet
`element`	`product` (test and live)
`description`	`Hybris-Adobe-File` (test and live)
`email_address`	`mulesoft@sgbd.co.uk` (test and live)
`check_divisions`, `export_results`, `overwrite_conflicts`	`true`
`header`	Column names read from the tab file, after the `## SC` lines

`rsid_list`	Test `saintgobaindistributiondev`. Live `saintgobaindistributionprod`.

Create returns the **job id**. Upload sends the tab file for that job, without the `## SC` lines. Commit runs only after the upload succeeded.

Status is a read after a successful commit. The daily Mule steps in the analysis sheet do not call status; the status route is a separate endpoint on adobe-cloud-sapi. This Function reads 2.0 job status for the log only. A status failure does not undo a commit that already succeeded. `GET /status` on adobe-cloud-sapi is that app's health check and is not part of this Function.

Authentication matches the token call in the analysis sheet, which is already Adobe IMS rather than the JWT described in the 2021 LLD. The Function requests a token at the start of the run and refreshes it when it is expired, and sends it on create, upload, commit, and status.

Token detail	Value
URL	`https://ims-na1.adobelogin.com/ims/token/v3`
Method	`POST`, `multipart/form-data`
`grant_type`	`client_credentials`
`scope`	`openid`, `AdobeID`, `additional_info.projectedProductContext`
Client id	One id for test and live, from Key Vault. The analysis sheet records the value; this page does not.

The sheet spells the middle scope value `AdobelD`. Confirm the live property before coding that string. A token failure stops the run before create. Upload and commit are not called.

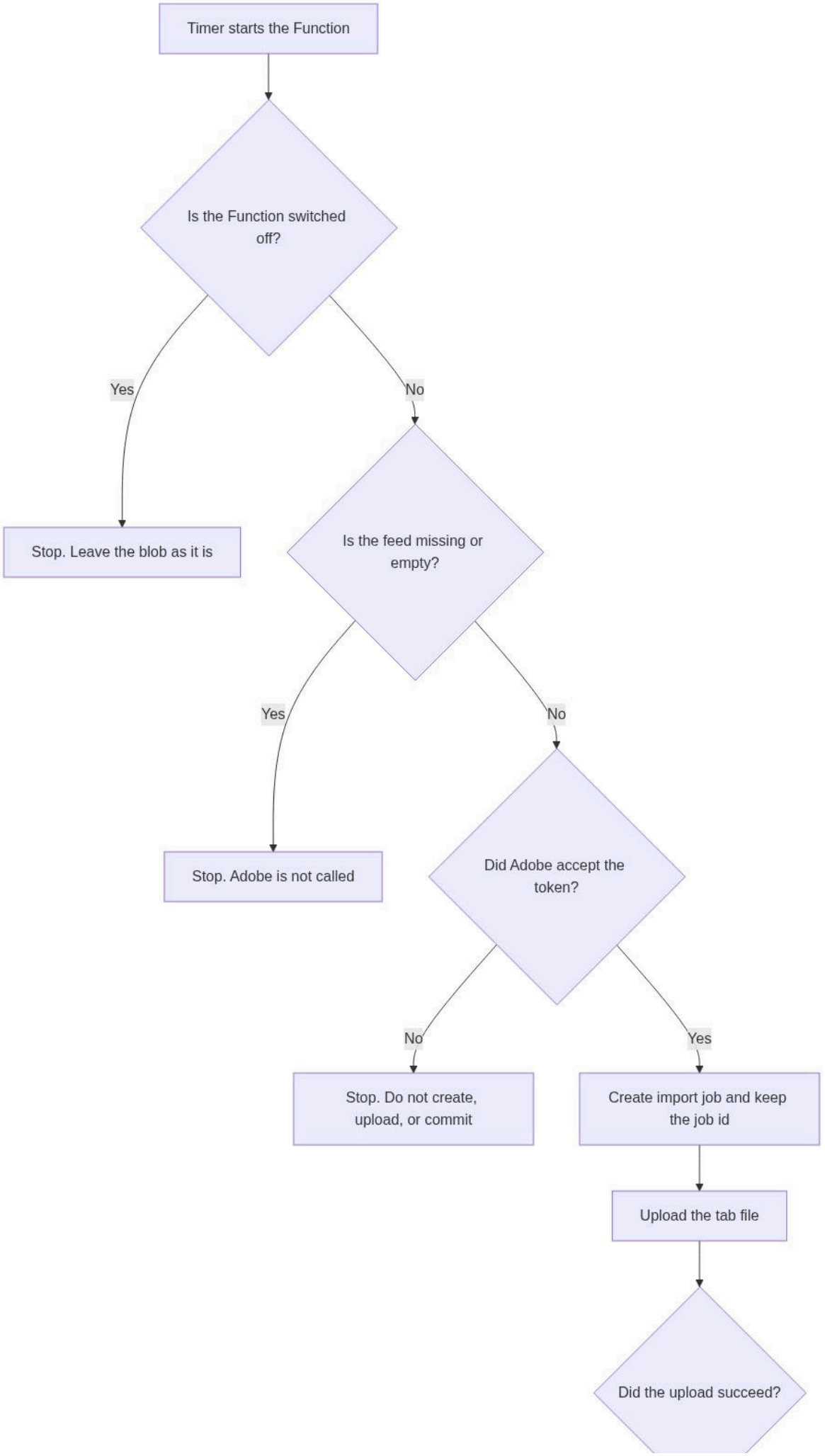
Client id, client secret, and the organisation details live in Key Vault. The analysis workbook contains a client secret in the token example. Do not copy it here, into source, or into Confluence.

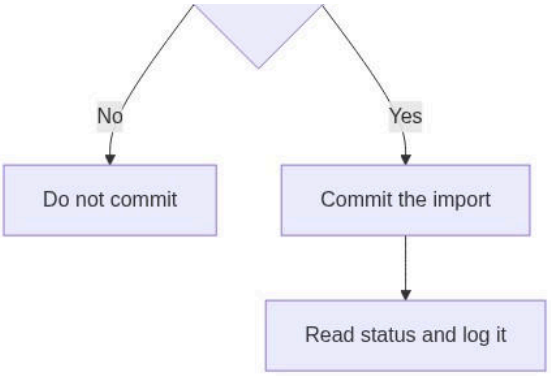
Synchronization Layer

One timer on `STUK.ProductService.Processor.Functions` runs the import. The service method is on `STUK.ProductService.Processor`.

Run outcomes

These outcomes are log entries. There is no HTTP caller waiting on this timer.


Timer starts the Function
Is the Function switched off?
Yes
No
Stop. Leave the blob as it is
Is the feed missing or empty?
Yes
No
Stop. Adobe is not called
Did Adobe accept the token?
No
Yes
Stop. Do not create, upload, or commit
Create import job and keep the job id
Upload the tab file
Did the upload succeed?



Mermaid source (paste into mermaid.live)

```
%% confluence-attachment: lld-classification-outcomes.png
%%{init: {"theme": "default", "flowchart": {"htmlLabels": true}}}%%
flowchart TD
  start[Timer starts the Function] --> disabled{Is the Function switched off?}
  disabled -->|Yes| stopOff[Stop. Leave the blob as it is]
  disabled -->|No| load{Is the feed missing or empty?}
  load -->|Yes| stopEmpty[Stop. Adobe is not called]
  load -->|No| token{Did Adobe accept the token?}
  token -->|No| stopToken[Stop. Do not create, upload, or commit]
  token -->|Yes| create[Create import job and keep the job id]
  create --> upload[Upload the tab file]
  upload --> upOk{Did the upload succeed?}
  upOk -->|No| noCommit[Do not commit]
  upOk -->|Yes| commit[Commit the import]
  commit --> status[Read status and log it]
```

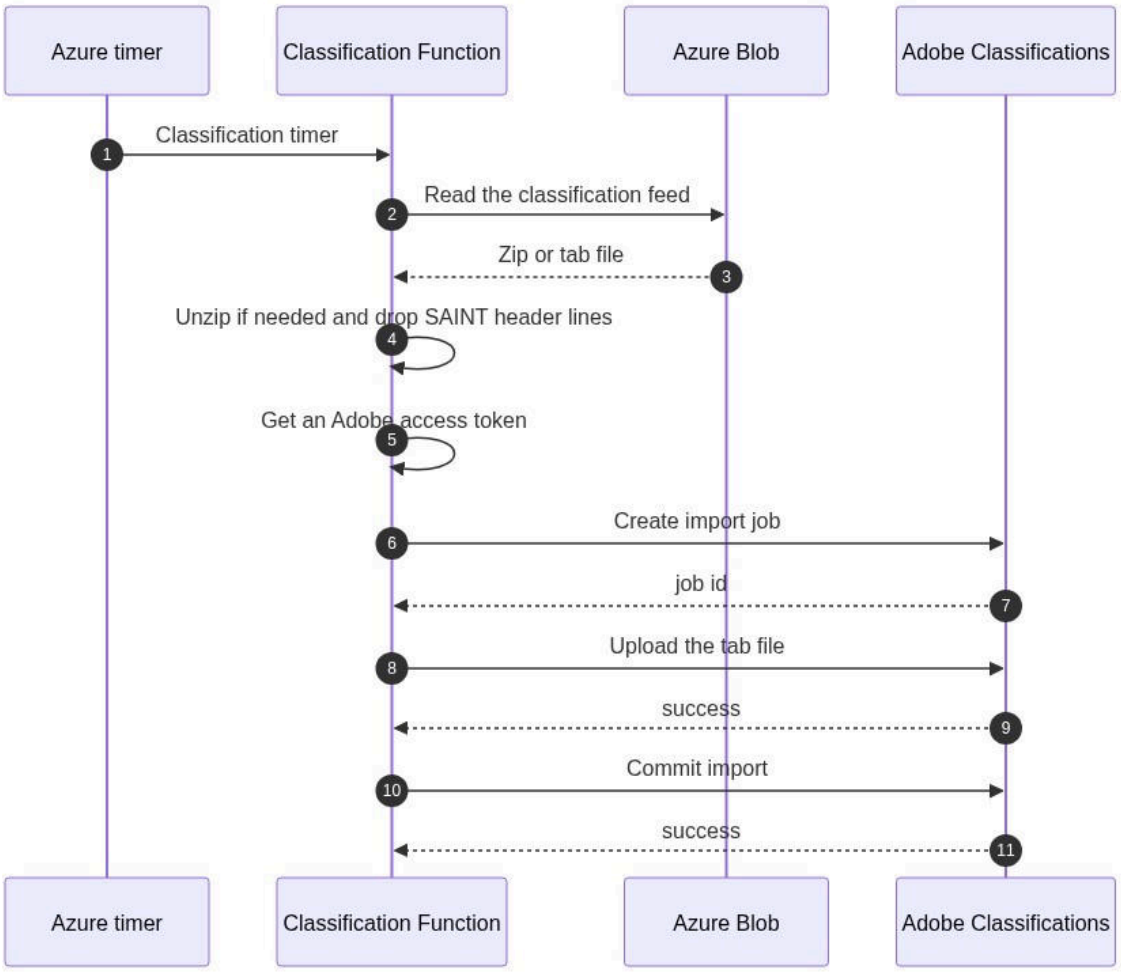
Outcome	What happens
Switched off	`DisableAdobeClassificationProcessor` is set. The run stops. The blob stays. Adobe is not called.
Missing or empty	No blob, empty bytes, header only, or a zip with no tab. Log that and stop. Adobe is not called.
Token failure	Log the failure. Do not create, upload, or commit. The run is a failure, not a success.
Create failure	Do not upload or commit.
Upload failure	Do not commit. An uncommitted Adobe job may remain. The next day's file is a new create. This design does not replay a partial upload.
Commit success	Log the job id. Then read status for the log.
Recovery	SAPCC overwrites the blob on the next drop. There is no row to reset and no automatic replay. A same-day rerun is an operator

	starting the Function again while that blob is still the full file.

Adobe HTTP 400 is a bad file: do not commit. Adobe HTTP 5xx or a timeout on upload is the same outcome for this feed: do not commit. The approved LLD does not define a retry count, because the file is a full daily drop.

Daily classification import

Happy path. Branches (disabled, empty, token failure, upload failure) are the flowchart above.



Mermaid source (paste into mermaid.live)

```
%% confluence-attachment: lld-daily-import.png
sequenceDiagram
    autonumber
    participant Timer as Azure timer
    participant Fn as Classification Function
    participant Blob as Azure Blob
    participant Adobe as Adobe Classifications

    Timer->>Fn: Classification timer
    Fn->>Blob: Read the classification feed
    Blob-->>Fn: Zip or tab file
    Fn->>Fn: Unzip if needed and drop SAINT header lines
    Fn->>Fn: Get an Adobe access token
    Fn->>Adobe: Create import job
```

```
    Adobe-->>Fn: job id
    Fn->>Adobe: Upload the tab file
    Adobe-->>Fn: success
    Fn->>Adobe: Commit import
    Adobe-->>Fn: success
```

Step	What happens
1	Azure timer starts `AdobeClassificationProcessor`.
2	Read the configured blob.
3	Blob returns the zip or the tab file.
4	Unzip when the blob is a zip and take `adobe-products-feed.tab`. Drop lines that start with `## SC`.
5	Get an Adobe IMS access token. Refresh if it expires before the run finishes.
6	`createApiJob` for the configured dataset id.
7	Adobe returns the job id.
8	`uploadFile` sends the tab file without the `## SC` lines.
9	Adobe returns success for the upload.
10	`commitApiJob` with the job id.
11	Adobe returns success. The Function logs the job id and may read job status.

The blob is still there after step 11.

How to Test

Compare a 2.0 job from this Function with the tab file SAPCC wrote. Do not use a live Mule 1.4 run as the oracle: those endpoints were retired on 31 August 2026.

SMEs for whether this MuleSoft replacement is acceptable:

- **Tommy McGrath** (SAPCC) and **Tom Wells** (Adobe), named as UAT contacts on STUK-11121.
- **Jordan Cowley**, named on adobe-cloud-sapi as the contact for the Adobe Developer portal. That page says he is the only person with access to reset the OAuth token. He has confirmed that this analytics data is still needed, and that the production import has been broken since August 2026, consistent with the 1.4 end of life.

The classification LLD leaves Consuming Applications blank. The AEC specification names the consumer: Adobe Analytics product reports, with Product Title and Category filled by the classification upload. The SMEs above are still the people who confirm that the current file is acceptable. A completed job is not that confirmation.

Integration tests use a stub Adobe 2.0 API and a fixture blob. The workshop note that only Populate is required is widened here to create, upload, and commit, because upload and commit decide whether a job is left open.

A successful commit only queues the file. To confirm Adobe processed the job, use either of these after UAT runs:

- Analytics UI: **Components → Classification sets → Jobs**. Filter **Job Type** to Import and **Source** to Direct API Upload. Success is status **Completed**. The row shows **Total Lines** and **Modified Lines**. Hover the alert on **Failed validation** or **Failed processing**. The same row can download the file for that job. See Classification Jobs Manager.
- Classifications 2.0: `GET /{globalCompanyId}/classifications/job/{jobId}` until `state` is `completed`, or `failed_validation` / `failed_processing`. A completed import includes `totalLines`, `noeffectLines`, and a history `message` (Adobe's example is `Successfully imported 5/5 records.`). Commit's HTTP success is not this state.

The live Mule daily steps do not read job status, and this design does not check every data row. Whether every row in today's feed is processed, and whether anyone monitors that, is still open with the SMEs.

What a successful replacement looks like in Analytics

A completed job means Adobe accepted the file. The replacement has worked for the reports when a product key from that file shows the same classification values in the report suite.

The AEC field table (user story SGEP-3365; product-group underscores in SGEP-3366) says how SAPCC should fill the file:

Field	How that page says to populate it
Key	Hybris SKU / article number. This is the same id the hit sends as the product.
Product Title	Hybris Identifier
Product Brand	Hybris Brand
Product Type	ICON Product Type

Product Group	ICON Product Group, with underscores removed
Legacy Product Group	ICON Legacy Product Group
Transaction Type	`sale` or `hire`
Availability Assignments	Availability group ids, separated by commas
Product Status	ICON Product Status
Product URL, Product Thumbnail URL	URLs on the brand site. The page names the Jewson store.
Product Category Full Path, Levels 1–5	Supercategory path, split on `>`. The page names the Jewson store.

The file SAPCC writes today is not that table. The 5 October 2026 sample, and the header on the approved Mule create-import example, are: Key, Product Title, Product Brand, Product Type, Product Group, Legacy Product Group, Transaction Type, Availability Assignments, Product Status, Product URL, Product Thumbnail URL, Available Online. Category columns are absent. Available Online is extra. Product Group in the sample still contains underscores (`C_A_D_D_D_J_`). The Function uploads that file. It does not rebuild columns to match the older AEC table.

The AEC example keeps the `## SC` lines and says not to remove them. That instruction is for the FTP SAINT importer. This Function drops those lines, matching what Mule sent to the API.

After a UAT job reaches `completed`, pick a key from the uploaded tab file. In the report suite for that run, break the product down by the classification columns that are in the file. For the sample key `DFLBM838`, Product Title should be `Jewson Framed Ledged & Braced Door 838 x 1981mm`, with the other values from that same row (brand Jewson, type E, transaction type sale, availability `jewson-availability`, status A, and the URL, thumbnail, and Available Online values). Allow for Adobe's delay between a completed job and the values appearing on reports.

An empty Category breakdown after a completed job matches the current file, which has no category columns. It is a gap against the AEC spec, and it is a question for the SMEs, not by itself a failed import. Transaction Type on Product Details Pages is also eVar36 from the data layer, so a sale or hire value on a hit is not proof the classification file was imported. The FTP account and the `.fin` file are not the acceptance test.

Scenario 1.1 Create, upload, commit

Given a tab or zip fixture with a header and at least one data row

When the processor runs with a valid token and a configured dataset id

Then create is called once and returns a job id

And upload is called once with that job id and the tab bytes, without any `## SC` line

And commit is called once with that job id

And the blob is not deleted

Scenario 1.2 Status after commit

Given a successful commit

When job status is called

Then the stub returns job status

And the import is not changed by that call

Scenario 1.3 No feed file

Given the container has no classification blob

When the processor runs

Then create, upload, and commit are not called

Scenario 1.4 Empty file

Given a blob that is empty, header-only, or a zip with no tab member

When the processor runs

Then create, upload, and commit are not called

And no job is committed

Scenario 1.5 Upload failure does not commit

Given a valid tab file and a stub that fails the upload

When the processor runs

Then create was called

And commit is not called

Scenario 1.6 Token failure

Given the token request fails

When the processor runs

Then create, upload, and commit are not called

And the run is a failure

Scenario 1.7 Disable flag

Given `DisableAdobeClassificationProcessor` is set

When the timer fires

Then the blob is not read and Adobe is not called

Scenario 1.8 Cutover does not stop shared apps

Given this Function is live for the UAT report suite

When Mule is switched off for this feed

Then only the CloudHub schedule `product-feed-papi-adobeFlow_Scheduler` (`product-feed-papi-adobeFlow`) is disabled

And other `product-feed-papi` schedules stay enabled

And `stark-hr-azure-storage-sapi` stays up for `MUL-CONCUR`

Risks

- **Adobe Analytics 1.4 is past end of life.** Adobe retired the 1.4 APIs and WSSE authentication on 31 August 2026. Classifications is on that list, and Experience League states the endpoints are no longer accessible. The live Mule SAPI still targets `api.omniture.com` 1.4. Jordan Cowley has confirmed that the production import has been broken since August 2026, which matches that retirement, and that the analytics data is still needed. Mitigation: this Function calls Classifications 2.0 only. Do not implement `CreateImport` , `PopulateImport` , or `CommitImport` .
- **Shared CloudHub apps.** `product-feed-papi` also serves other feeds. `stark-hr-azure-storage-sapi` serves `MUL-CONCUR` . Mitigation: disable the adobe Flow schedule only. Do not stop those applications for this id.

- **Zip versus tab.** The process LLD says both `adobe-products-feed.tab` and a zip response. The analysis sheet names a single tab at `hybris/export`. Mitigation: accept either, and confirm the container on S2/P2 before the first non-prod run.
- **SAINT preamble.** The sample feed starts with `## SC` lines. This design drops them before upload so the file matches what Mule sent to Adobe. Mitigation: confirm 2.0 accepts that stripped tab, and that it does not require the raw SAINT preamble.
- **Classification dataset.** 2.0 imports into an existing dataset. The API does not create that dataset, and it does not take the old `rsid_list` / `element` / `header` create body. The analysis sheet gives the report suites `saintgobaindistributiondev` (test) and `saintgobaindistributionprod` (live). Mitigation: map each suite to a dataset id in the Adobe Analytics UI before the first run.
- **IMS scope spelling.** The analysis sheet writes the scope as `AdobelD`. Mitigation: confirm the live Mule property before using that spelling.
- **Upload size.** The OpenAPI description of `uploadFile` says 50 MB. The sample feed is about 11 MB. Mitigation: confirm the limit still holds if the catalogue grows past 50 MB.
- **Secret in the analysis workbook.** `docs/product-classification/Phase2_PRODUCT-CLASSIFICATION-Analysis_Sheet_v1.0.xlsx` contains a client secret. Mitigation: do not publish those cells; keep the secret in Key Vault and rotate it if the workbook has been shared.
- **Upload operation is untyped.** `uploadImportFile` has no request body in `classification.json`, so NSwag will not generate the file parameter. Mitigation: hand-write that one call.
- **Uncommitted Adobe job.** An upload failure leaves a job that was created and not committed. Mitigation: do not commit; log the job id; the next full file is a new create.
- **Adobe test environment.** The 2021 LLD says a test environment for the classifications API was not available. Mitigation: the integration tests use a stub; UAT needs an agreed 2.0 dataset before job-id checks against the real API.
- **Dual run.** If the Mule timer and this Function both import the same blob, Adobe gets two jobs when Mule can still reach Adobe. Mitigation: disable `product-feed-papi-adobeFlow_Scheduler` when this Function is the writer for that environment.
- **Acceptance is a report breakdown, and the file has drifted from the analytics spec.** Product Details Pages sources Product Title and Category from Classification Upload. The current feed has no category columns and adds Available Online, and the sample Product Group still contains underscores. Mitigation: after UAT, match one uploaded key to the columns that are in the file. Confirm with Tommy McGrath, Tom Wells, and Jordan Cowley that this column set is what the reports still need. Do not treat the AEC page's FTP and `.fin` steps as the test.

Appendix: Included MuleSoft Replacements

Stop the schedule, not the whole application.

Live app	What this slice replaces	What stays
`sgbd-live-product-feed-papi-v1`	Schedule `product-feed-papi-adobeFlow_Scheduler` on flow `product-feed-papi-adobeFlow` (`0 0 6 * * ? *`)	Every other product-feed schedule on that app
`sgbd-live-adobe-cloud-sapi-v1`	Create, populate, commit, status, and the hourly token flow, for this feed only. The replacement uses Classifications 2.0, not these 1.4 routes	The app, until nothing else calls it. The approved LLD lists `product-feed-papi` as the consumer
`stark-live-stark-hr-azure-storage-sapi-v1`	The classification blob download only, and only by no longer calling it from this flow	The app itself, including `MUL-CONCUR`
`sgbd-live-product-papi-v1`	Nothing. Undeployed	n/a

Matrix id: `MUL-PRODUCT-CLASSIFICATION`. Integration UAT: STUK-11121. Timer build ticket: STUK-11303.

Appendix: Glossary

Classification feed

The full Adobe product file SAP Commerce Cloud writes to Azure Blob once a day, at `hybris/export` / `adobe-products-feed.tab`. It is a SiteCatalyst SAINT file: `## SC` lines, then a `Key` header, then data rows. Sometimes delivered as a zip that contains that tab.

Classification Function

The timer-triggered Azure Function `AdobeClassificationProcessor` on `STUK.ProductService.Processor.Functions`. It reads the feed and imports it into Adobe.

Adobe Classifications

Adobe Analytics Classifications API 2.0 on `https://analytics.adobe.io/api`: create job, upload the tab file, commit job. Documentation: Analytics Classifications API. The retired 1.4 methods on `api.omniture.com` are not called.

Job id

The id Adobe returns from `createApiJob`. Upload and commit both send it. It is not stored in SQL in this design; it is logged.

Full feed

The whole product classification file for that day. The next drop replaces it. This design does not replay a failed upload.

Product feed PAPI

`sgbd-live-product-feed-papi-v1`. The adobe Flow on this app is the timer being replaced. Other flows on the app are out of scope.

Adobe cloud SAPI

`sgbd-live-adobe-cloud-sapi-v1`. System API the process app calls today. This design calls Adobe directly instead.

HR Azure storage SAPI

`stark-live-stark-hr-azure-storage-sapi-v1`. Downloads the blob today. This design uses the Azure SDK. The SAPI remains for other integrations.

Appendix: Original diagrams under source control

Diagram sources for mermaid.live live under `docs/product-classification/design-diagrams/mermaid/`. Confluence does not render a mermaid code block as a picture; publish attaches a PNG and keeps the source in an expand.

```
cd tools/product-classification/confluence-crawl
node render-mermaid.mjs
node md-to-storage.mjs
$env:CONFIRM_CONFLUENCE_PUBLISH = "1"
node update-draft-product-classification-page.mjs
```

Run the last command only after this markdown is accepted. It creates or updates the child page **DRAFT - Product Classification** under Phase 1.5.

Diagram	File
High Level Design	`docs/product-classification/design-diagrams/mermaid/hld-product-classification.mmd`
Run outcomes	`docs/product-classification/design-diagrams/mermaid/lld-classification-outcomes.mmd`
Daily classification import	`docs/product-classification/design-diagrams/mermaid/lld-daily-import.mmd`

ERP / Backoffice Systems

- SAP Commerce Cloud (writes the classification feed to Azure Blob)
- Adobe Marketing Cloud / Adobe Analytics classifications (consumes the import)
- Azure Blob on the SAP cloud storage account (S2 test, P2 prod). Path from the analysis sheet: `hybris/export` / `adobe-products-feed.tab`. Container name still to confirm.

Links and Resources

- Parent page: [Phase 1.5]
- Analytics file contract: [Adobe Analytics Product Classification File] (version 9), under [Solution Design]

- Report specification beside that file: Analytics Solution Design (version 4). Product Title and Category on Product Details Pages are Classification Upload.
- Approved process LLD: product-feed-papi (adobe classification integration)
- Approved system LLD: adobe-cloud-sapi
- Storage SAPI LLD: stark-hr-azure-storage-sapi
- Adobe Analytics 2.0 Classifications OpenAPI: classification.json
- Adobe Analytics 1.4 end of life (31 August 2026): retired APIs
- Migration guide: Migrating to the 2.0 API
- Integration ticket: STUK-11121
- Timer ticket: STUK-11303
- Analysis sheet: `docs/product-classification/Phase2_PRODUCT-CLASSIFICATION-Analysis_Sheet_v1.0.xlsx` (sheet `PRODUCT-CLASSIFICATION-APIs`)
- Sample feed: `docs/product-classification/sample/adobe-products-feed.tab`
- Local crawl of the pages above: `docs/product-classification/confluence/`